

# RescueADI: Adaptive Disaster Interpretation in Remote Sensing Images With Autonomous Agents

Zhuoran Liu, Danpei Zhao<sup>ID</sup>, *Member, IEEE*, Bo Yuan<sup>ID</sup>, *Graduate Student Member, IEEE*,  
and Zhiguo Jiang<sup>ID</sup>, *Member, IEEE*

**Abstract**—Current methods for disaster scene interpretation in remote sensing images (RSIs) mostly focus on isolated tasks such as segmentation, detection, or visual question-answering (VQA). However, these methods often fail to provide comprehensive and actionable insights, particularly in scenarios that demand the integration of multiple perception methods and specialized tools to address complex, multilayered challenges in geophysical disaster analysis. To fill this gap, this article introduces adaptive disaster interpretation (ADI), a novel task designed to solve requests by planning and executing multiple sequentially correlative interpretation tasks to provide a comprehensive analysis of disaster scenes. To facilitate research and application in this area, we present a new dataset named RescueADI, which contains high-resolution RSIs with annotations for three connected aspects: planning, perception, and recognition. The dataset includes 4044 RSIs, 16 949 semantic masks, 14 483 object bounding boxes, and 13 424 interpretation requests across nine challenging request types. Moreover, we propose a new disaster interpretation method employing autonomous agents driven by large language models (LLMs) for task planning and execution, proving its efficacy in handling complex disaster interpretations. The proposed agent-based method solves various complex interpretation requests such as counting, area calculation, and path finding without human intervention, which traditional single-task approaches cannot handle effectively. Experimental results on RescueADI demonstrate the feasibility of the proposed task and show that our method achieves an accuracy 9% higher than existing VQA methods, highlighting its advantages over conventional disaster interpretation approaches.

**Index Terms**—Autonomous agent, disaster interpretation, large language model (LLM), remote sensing images (RSIs).

## NOMENCLATURE

|     |                                                |
|-----|------------------------------------------------|
| RSI | Remote sensing image.                          |
| AI  | Artificial intelligence.                       |
| LLM | Large language model.                          |
| VLM | Vision language model.                         |
| VQA | Visual question answering.                     |
| IoU | Intersection over union, an evaluation metric. |
| AP  | Average precision, an evaluation metric.       |

Received 18 October 2024; revised 13 December 2024 and 9 January 2025; accepted 16 January 2025. Date of publication 22 January 2025; date of current version 27 February 2025. This work was supported by the National Natural Science Foundation of China under Grant 62271018. (*Corresponding author: Danpei Zhao.*)

Zhuoran Liu is with the Image Processing Center, School of Astronautics, Beihang University, Beijing 100191, China (e-mail: liuzhuoran@buaa.edu.cn).

Danpei Zhao, Bo Yuan, and Zhiguo Jiang are with the Image Processing Center, School of Astronautics, Beihang University, Beijing 100191, China, and also with the Tianmushan Laboratory, Hangzhou 311115, China (e-mail: zhaodanpei@buaa.edu.cn; yuanbobuaa@buaa.edu.cn; jiangzg@buaa.edu.cn).

Digital Object Identifier 10.1109/TGRS.2025.3532594

VR Valid rate, an evaluation metric.

ADI Adaptive disaster interpretation, the proposed task form.

## I. INTRODUCTION

DISASTER detection based on RSIs provides accurate, efficient, and wide-area disaster assessment and situation judgment [1]. With the development of RSI interpretation technology, deep learning models have played an important role in various tasks including land resource statistics [2], urban planning [3], disaster monitoring [4], etc. Deep learning models have also been applied to pre-disaster monitoring [5] and post-disaster recovery [6]. Especially in disaster damage assessment, neural networks can quickly extract disaster-related information from RSIs and provide it to rescuers for analysis to help subsequent disaster assessment and response.

In terms of task formation, most of the existing disaster monitoring models are focused on isolated tasks, such as segmentation [6] and scene classification [7]. Segmentation tasks can be further divided into semantic segmentation [6] and change detection [8]. Concretely, semantic segmentation takes an input image and predicts pixel-level damage level, while change detection utilizes two images of the same location with different dates and predicts a pixel-level mask to indicate the change of semantics. Scene classification [7] aims to categorize the type of disaster or the level of damage at the image level. These perception tasks are good at extracting information from the input image. Numerous techniques, including attention [4], pyramid pooling [9], weak supervision [10], and the integration of prior knowledge [11], have been developed to enhance the accuracy of perception systems. However, the extracted information is not in the form of natural language and requires further recognition to provide effective guidance to the rescue missions. To cope with more flexible application scenarios, VQA has emerged and become a popular research direction for disaster assessment [12], [13]. VQA methods respond to user input in natural language and give answers in text form, providing highly abstract feedback and reducing the understanding cost. Recent developments in LLMs [14] and VLMs [15] have also further extended the boundary of VQA models. However, the VQA task does not explicitly produce intermediate perception results and thus lacks transparency. When it comes to questions related to

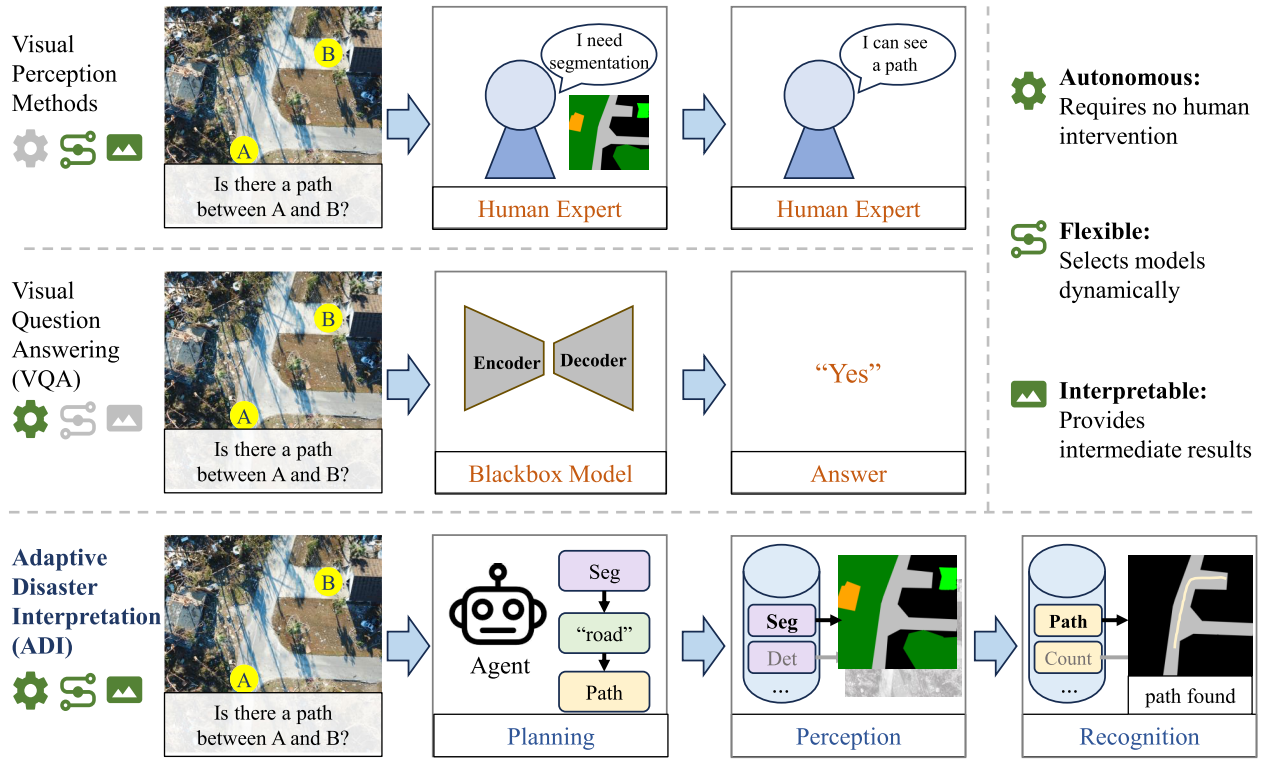


Fig. 1. Proposed ADI integrates planning, perception, and recognition without the need of human intervention, along with clear intermediate results.

quantitative analysis, even the largest LLMs cannot avoid the problem of hallucination [16], making it difficult for them to output realistic and accurate numbers. In addition, VQA methods are still limited, as the task form only supports answering questions but not responding to human instructions for executing tasks, such as performing segmentation on a given image.

Recently, LLMs have emerged to approach human-level intelligence [17], [18], [19]. LLM-driven autonomous agents have become increasingly capable of performing complex tasks with minimal human intervention [20], [21]. The key to the flexibility of LLM-driven autonomous agents is the ability to make plans to utilize multiple tools. Similarly, to systematically survey and analyze remote sensing data, it is required to perceive the RSI of the disaster from multiple aspects [22]. Especially in disaster scene interpretation, human experts often combine the results of several different models or run one model after another sequentially in order to get the correct results [23]. Inspired by this, we propose ADI as a new form of task where an interpretation system is required to do planning according to the user's request about the disaster scene and invoke a series of modular sub-tasks to get an accurate and detailed answer.

As shown in Fig. 1, ADI unifies the need to perform individual interpretation tasks and the VQA tasks that require a comprehensive understanding of the disaster scene. Compared to simply consolidating all sub-tasks into one, ADI models the connections between sub-tasks through planning. Different from existing tasks, ADI requires specialized interpretation of disaster scenes to be performed. For example, several

interpretation tasks need to be performed sequentially to determine whether rescuers can reach the damaged houses. The first step is to determine the damage to the houses in the area as well as the obstruction of the roads, which is the basic interpretation task that current disaster detection algorithms are concerned with. Next, it is also necessary to determine whether the damaged houses can be reached, which is a high-level planning task. Finally, the results of each step are summarized into a structured text that is easy for humans to understand.

To support the research on the new task, we create RescueADI dataset based on high-resolution RSIs. Our dataset consists of 13 424 questions in nine different aspects that are difficult to accomplish through existing single-model methods. To the best of our knowledge, this is the first disaster interpretation dataset covering multiple complex disaster interpretation scenarios for autonomous agents. Additionally, we propose an effective method to tackle the novel ADI task by constructing an LLM-based autonomous agent framework, utilizing LLM's planning capabilities. Experimental results validate the feasibility of the ADI task and provide a solid basis for future research.

The contribution of this article can be summarized as follows.

- 1) We propose ADI, a new task where autonomous agents use sequential modular tools to interpret complex user queries about disaster scenarios and provide more understandable responses. The task form allows adaptive interpretation based on different requests, which existing single-task frameworks cannot handle.

- 2) To cope with ADI, a new dataset is presented named RescueADI. To our knowledge, this is the first time that various annotations for RSIs have been incorporated to challenge and enhance disaster interpretation tasks for autonomous agents.
- 3) We design an autonomous agent-based framework to address the ADI problem. The proposed method utilizes LLMs for planning, validating the feasibility of ADI for complex disaster scenarios. The framework generalizes better and achieves 9% accuracy improvement compared to conventional VQA methods.

## II. RELATED WORKS

### A. Remote Sensing Interpretation in Natural Disaster Scenarios

Currently, many datasets for disaster scenarios are available in the form of basic computer vision tasks such as semantic segmentation, instance segmentation, change detection, and scene classification.

Most commonly, detecting damaged areas on post-disaster images can be viewed as a segmentation task, where class labels are assigned to each pixel of the image. Semantic segmentation is a well-studied task in general RSIs, with many datasets available [27], [28], [29]. Different from general semantic segmentation datasets, a disaster detection dataset uses disaster scenarios as a source of images and provides more detailed annotations of the affected area. RescueNet [6] provides a detailed post-disaster imagery dataset captured following Hurricane Michael, with a focus on semantic segmentation. It includes comprehensive pixel-level annotations across various classes including fine-grained building damage levels, roads, water, and vegetation. The ISBDA [25] dataset provides instance-level building damage masks within user-generated aerial videos from social media. This dataset focuses on quantitative model evaluation, offering new perspectives in the application of aerial video analysis for assessing building damages in post-disaster scenarios.

Detecting damaged areas given pre- and post-disaster images can be viewed as a change detection task. Change detection involves monitoring semantic changes in the surface over time and is commonly used in land surveys and environmental monitoring studies [30]. In natural disaster scenarios, the xBD dataset [8] offers a unique collection of both pre- and post-event satellite imagery, designed to support change detection and building damage assessment for disaster recovery efforts.

Scene classification is also a viable option for post-disaster assessment. In scene classification datasets, a label is assigned to each image to indicate the category of the image. Unlike object classification datasets of natural images [31], [32], [33], [34] that focus on the foreground object in the image, scene classification in RSIs takes into account the background information. Most scene classification datasets for RSI focus on normal scenarios [35], [36], [37], while AIDER [7], a dataset specially crafted for various disaster scenarios, provides invaluable insights into disaster response and recovery efforts.

Despite the rapid development of RSI datasets in disaster scenarios, the above datasets are designed for specific tasks and lack flexibility. Additional work by experts is still needed to perform these tasks and convert the output of each task to information that can guide post-disaster relief efforts. Therefore, a more integrated system that can make task execution plans and synthesize information from different tasks is crucial for enhancing decision-making in post-disaster situations.

### B. VQA for Remote Sensing

VQA [12], [38], [39], [40], [41] is a challenging task that aims to bridge the gap between visual information and natural language understanding. Specifically, the objective of VQA is to answer questions about visual content in images with precise textual responses. With the development of LLMs, researchers have combined vision models with language models to construct VLMs such as Visual-GLM [42] and LLaVA [43], which surpass the performance of traditional VQA models in various datasets. Efforts have also been made to adapt VQA to remote-sensing images. RSVQA [26] proposes a system and a corresponding dataset to extract textual answers to given questions from RSI. Prompt-RSVQA [44] enhances the answer accuracy by providing a language model with contextual prompts. GeoChat [45] finetunes the LLaVA [43] to answer questions on RSIs. As for disaster detection, FloodNet [24] and FloodNet+ [46] combine the semantic segmentation task, the scene classification task, and the VQA task into one dataset, providing a more comprehensive understanding of disaster scenes. The dataset provides multiple question-answer pairs along with a segmentation map for each image. However, there is still a lack of modeling of the relationship between tasks and questions. These tasks are considered independent parts and are treated separately. In addition, predicting the answers to numerical questions in RSI is still a difficult task for end-to-end models as they do not explicitly perform basic perception tasks.

When human experts solve problems in disaster scenarios in remote sensing, not only do they answer questions, but they also make different plans to perform sub-tasks depending on the nature of the problem. Motivated by this, we extend the VQA framework from simply answering questions to planning sub-tasks to solve problems. In this article, we establish ADI, a new task form with the ability of planning and question answering, along with a novel dataset to support our research.

### C. Action Planning With Autonomous Agents

An autonomous agent refers to a program that is able to interact with the environment according to certain rules or policy functions and accomplish certain tasks [47], [48]. The recent development of LLMs has given new insights into the study of autonomous agents. Pretrained on a large amount of text, an LLM learns a large number of logical rules and common sense and is able to drive an agent to carry out more complex task planning and external interactions [21]. Camel [49] proposes that allowing an LLM to engage in role-playing and multiagent cooperation can enhance its ability to solve complex problems. ChatDev [50] utilizes multiple



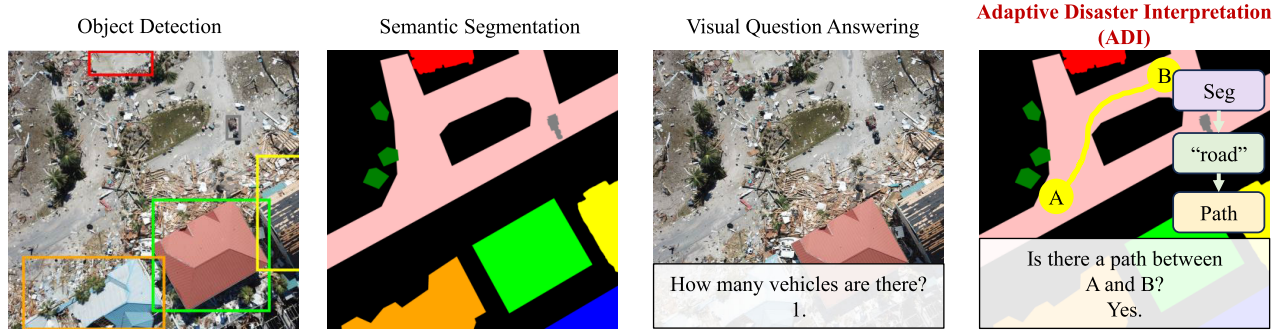


Fig. 2. Comparison between ADI and existing task forms. Instead of performing an isolated task, ADI models the connection between planning, perception, and recognition.

LLMs to play the roles of various positions in a game development company, realizing automated, stable, and reliable implementation of complex development requirements. DEPS [51] proposes an interactive task-planning framework that is able to reach difficult task goals in open-world game simulations. GITM [52] proposes a recursive tree-based task planning system that gradually disassembles complex tasks into simple ones. WebGPT [53] uses GPT3 as a planner, which continuously interacts with the search engine and answers the user's questions. ToolFormer [54] proposes external tools to be integrated into LLMs. VisualChatGPT [55] employs GroundingDINO [56] and SAM [57] as external tools to be invoked by the LLM and realizes the recognition and editing of input images according to the user's needs. HuggingGPT [58] packages a large number of models on an open-source modeling platform as tools that can be invoked by an autonomous agent, which is capable of completing complex task processes that require the combination of multiple models.

In remote sensing, pioneering work has been done to utilize the power of LLM in image interpretation [59]. However, existing methods are limited to natural scenes and lack specialized tools for disaster interpretation.

Despite the rapid progress in research on action planning, most developments are tailored for specific applications. Limited efforts have been dedicated to disaster interpretation, which demands specialized models, tools, and expertise. In this article, we propose a standardized formulation of ADI and introduce a novel dataset to validate action planning in disaster scenarios. Additionally, we develop a baseline method that leverages the planning capabilities of LLMs and incorporates specialized tools to enhance its adaptability for interpreting disaster scenarios in RSIs.

### III. TASK DEFINITION OF ADI

In this article, we introduce a novel task called ADI, designed to enhance our understanding of disaster scenarios through an agent-based approach. As demonstrated in Fig. 2, our framework differs from traditional tasks by integrating sub-task planning and question-answering capabilities. Unlike conventional tasks that are limited to specific forms such as semantic segmentation or visual question answering, ADI requires an agent that can dynamically adjust to the demands of the input request. For instance, it can perform detailed semantic segmentation of affected areas when required, while

also being able to answer contextual questions about the disaster, such as the extent of damage or the number of affected structures.

The input of ADI consists of an input image and a textual request. The input image provides visual information about the disaster scene while the request provides the objective of the interpretation task, which could be a request to perform a specific sub-task, a question to be answered, or both. The output of ADI can be defined from three connected aspects, as shown in Fig. 3.

- 1) The planning aspect asks an agent to create a plan according to the need of the input request. Each step of the plan belongs to a set of sub-tasks predefined by the dataset. The agent should perform every necessary sub-task while avoiding planning for sub-tasks that are not needed.
- 2) The perception aspect requires every selected sub-task to output accurate prediction results. For example, a segmentation sub-task involves assigning categories to each individual pixel within the input image, and an object detection sub-task entails the identification and localization of objects in the input image.
- 3) The recognition aspect demands the agent to understand and analyze the perception result and generate a direct response to the request in natural language.

Based on the above three aspects, a more specific definition of ADI is presented as follows. The general input of the task is defined as a tuple  $(I, Q)$ , where  $I \in \mathbb{R}^{C \times H \times W}$  is the input image and  $Q$  is the input request in natural language.  $C$  represents the number of channels in the image,  $H$  is the height, and  $W$  is the width of the image. The output of ADI is a structured response that integrates the results from the planning, perception, and recognition aspects. Formally, the output can also be represented as a tuple  $(P, R, A)$ , where

- 1)  $P$  is a sequence of planned sub-tasks,  $P = \{p_1, p_2, \dots, p_n\}$ , where each  $p_i$  belongs to a predefined set of sub-tasks  $\mathcal{P}$ . The agent must select and plan these sub-tasks based on the input request  $Q$ .
- 2)  $R$  is a set of results from the executed sub-tasks,  $R = \{r_1, r_2, \dots, r_n\}$ , corresponding to the sub-tasks in  $P$ . Each  $r_i$  is the output of the sub-task  $p_i$ , which could include segmentation maps, object detection bounding boxes, or other form of basic perception tasks.

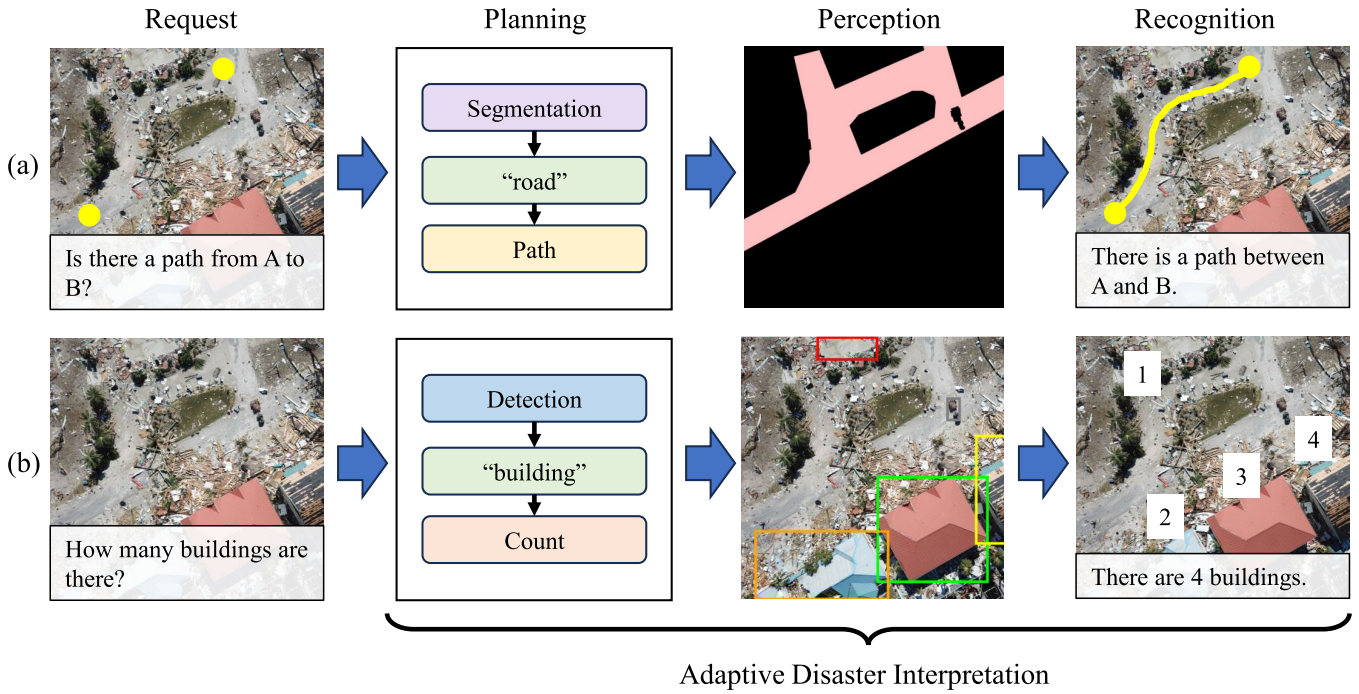


Fig. 3. Definition and examples of ADI. The task has three stages: planning, perception, and recognition. The planning stage dynamically schedules the following perception and recognition, enabling flexible interpretation of disaster scenarios. (a) Demonstration of a path-finding request. This requires a combination of a segmentation model and a path-finding algorithm to get the correct result. (b) Demonstration of a counting request. This requires a combination of a detection model and counting of the detection results to get the correct answer.

- 3)  $A$  is the final natural language answer generated in response to the input request  $Q$ . This answer should be coherent, contextually relevant, and derived from the request  $Q$  and the perception results  $R$ .

The overall objective of ADI is to maximize the accuracy of  $P$ ,  $R$ , and  $A$  with respect to the input tuple  $(I, Q)$ .

The following metrics are employed to evaluate the performance of an agent on ADI.

- 1) *Planning Accuracy*: To measure the correctness of the planned sub-tasks  $P$  against a ground truth sequence of sub-tasks  $P^*$ , we treat the presence of each sub-task as a binary classification problem and utilize precision and recall metrics.
- 2) *Perception Accuracy*: Metrics commonly used in basic perception tasks are adopted to measure the perception accuracy of selected sub-tasks. For semantic segmentation sub-tasks, we use IoU to represent the accuracy of the segmentation outputs. For object detection sub-tasks, we employ mean AP (mAP) to verify the quality of the bounding boxes.
- 3) *Recognition Accuracy*: Evaluate the quality of the generated answer  $A$  by comparing it to the ground truth answer  $A^*$  using exact-match accuracy and GPTScore [58]. The former checks for identical answers, while the latter assesses semantic similarity to capture nuanced differences.

To summarize, ADI represents a comprehensive and dynamic task that challenges agents to integrate planning, perception, and recognition capabilities to effectively interpret and respond to complex disaster scenarios. This approach

aims to push the boundaries of current AI systems and foster advancements in disaster response technologies.

#### IV. DATASET

Existing datasets for individual tasks are not capable of providing a comprehensive understanding of disaster scenes necessary to accomplish the ADI task. Therefore, we construct RescueADI, a novel dataset designed to support the integrated task of sub-task planning, perception, and recognition. The dataset will be made accessible to the public online through our project page.<sup>1</sup> In this section, a detailed explanation of the creation of RescueADI is given.

##### A. The RescueADI Dataset

The proposed RescueADI dataset, designed for disaster scenes in RSIs for autonomous agents, distinguishes itself from existing datasets through its unique task format and comprehensive coverage of disaster interpretation tasks. The images of disaster scenes in our dataset are primarily sourced from the RescueNet dataset, covering the impacted area in Florida, USA, following Hurricane Michael's landfall on October 10, 2018. However, we have conducted additional data cleaning and developed methods to generate high-quality request-planning-answer annotations based on the original dataset. The RescueADI dataset is designed to support disaster interpretation requests, focusing on six categories of semantics and four fine-grained building damage levels. Additionally, it includes nine distinct types of requests, addressing the

<sup>1</sup>[https://xdedss.github.io/rescue\\_adi/](https://xdedss.github.io/rescue_adi/)

TABLE I

CAPABILITIES COVERED BY RESCUEADI. IN ADDITION TO PIXEL-LEVEL AND INSTANCE-LEVEL PERCEPTION, RESCUEADI ALSO COVERS PLANNING AND VARIOUS RECOGNITION TASKS, FULFILLING THE NEED OF THE PROPOSED ADI TASK. THE DATASET CONTAINS NOT ONLY CONVENTIONAL SEGMENTATION MASKS AND BOUNDING BOXES FOR PERCEPTION BUT ALSO TASK ANNOTATIONS FOR PLANNING AND QUESTION-ANSWERING ANNOTATIONS FOR RECOGNITION, WHICH WILL BE INTRODUCED IN DETAIL IN SECTION IV

| Task             | Planning | Perception  |                | Recognition       |                     |             |
|------------------|----------|-------------|----------------|-------------------|---------------------|-------------|
|                  |          | Pixel-level | Instance-level | Object Perception | Fine-Grained Damage | Rescue Path |
| xBD [8]          |          | ✓           |                |                   | ✓                   |             |
| FloodNet [24]    |          | ✓           |                | ✓                 |                     |             |
| ISBDA [25]       |          | ✓           | ✓              |                   |                     |             |
| RescueNet [6]    |          | ✓           |                |                   | ✓                   |             |
| RSVQA [26]       |          |             |                | ✓                 |                     |             |
| RescueADI (Ours) | ✓        | ✓           | ✓              | ✓                 | ✓                   | ✓           |

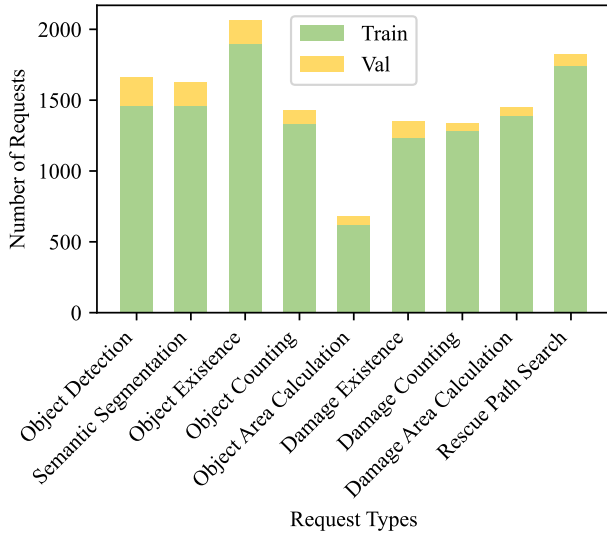


Fig. 4. Distribution of different request types in RescueADI.

fundamental needs and challenging scenarios encountered in remote sensing-based disaster interpretation.

As shown in Table I, compared to existing datasets, our dataset is the first one to cover the full spectrum of planning, perception, and recognition. To be specific, RescueADI contains 13 424 requests and answers on 4044 RSIs on disaster scenes, along with 16 949 semantic masks and 14 483 object bounding boxes. The fine-grained damage levels range from no damage to totally destroyed, and the categories for other semantics include water, vehicle, clear road, blocked road, pool, and trees. The types of requests comprise object detection, semantic segmentation, object existence, object counting, object area calculation, damage existence, damage counting, damage area calculation, and rescue path search. Fig. 4 illustrates the number distribution of tasks covered by our dataset. The dataset is split into 12 426 requests for training and 998 requests for validation.

### B. Request Formulation

In real-world disaster scenarios, the need for accurate and actionable information is emphasized. Responders require fast and precise data to make informed decisions that can save lives and mitigate damage. These needs can be categorized into several key areas.

- 1) *Basic Task Execution*: Fundamental-to-disaster response tasks include identifying critical objects (e.g., vehicles and buildings) or specific regions (e.g., blocked road and clear road) within an image. These basic identifications are crucial for initial assessments and resource allocation.
- 2) *Object Perception*: In disaster scenes, responders need to recognize and categorize various elements within a scene. This includes tasks such as determining the existence of objects, counting the number of objects, and calculating the area they occupy. Accurate object perception aids in understanding the complexity of the scene and planning appropriate responses.
- 3) *Fine-Grained Damage Assessment*: Detailed analysis of the extent and severity of damage is essential for prioritizing response efforts and resources. This includes tasks such as determining the existence of damage, counting instances of damage, and calculating the area affected by the damage. Fine-grained damage assessment provides a deeper insight into the disaster's impact and helps in formulating effective recovery strategies.
- 4) *Rescue Path Search*: In chaotic and hazardous environments, identifying safe and efficient routes for rescuers is vital. This involves analyzing the terrain, debris, and potential hazards to find paths that minimize risk and maximize response efficiency. Effective rescue path search can significantly improve the speed and safety of rescue operations.

Based on these critical needs, we have constructed requests in our dataset that aligns with these four general types. This systematic approach is derived from a thorough analysis of real-world disaster scenarios and considers the current limitations and capabilities of existing algorithms. By addressing these specific needs, our dataset provides a comprehensive tool for improving disaster response and management.

As illustrated in Fig. 5, nine subtypes are derived from these four general types. Basic task execution includes the demand to execute semantic segmentation or object detection on the given image. Object perception includes tasks such as determining the existence of objects, counting the objects, and calculating the area they occupy. For instance, object presence requests involve answering “yes/no” questions about the existence of certain objects, while object counting and area calculation requests require precise numerical answers.



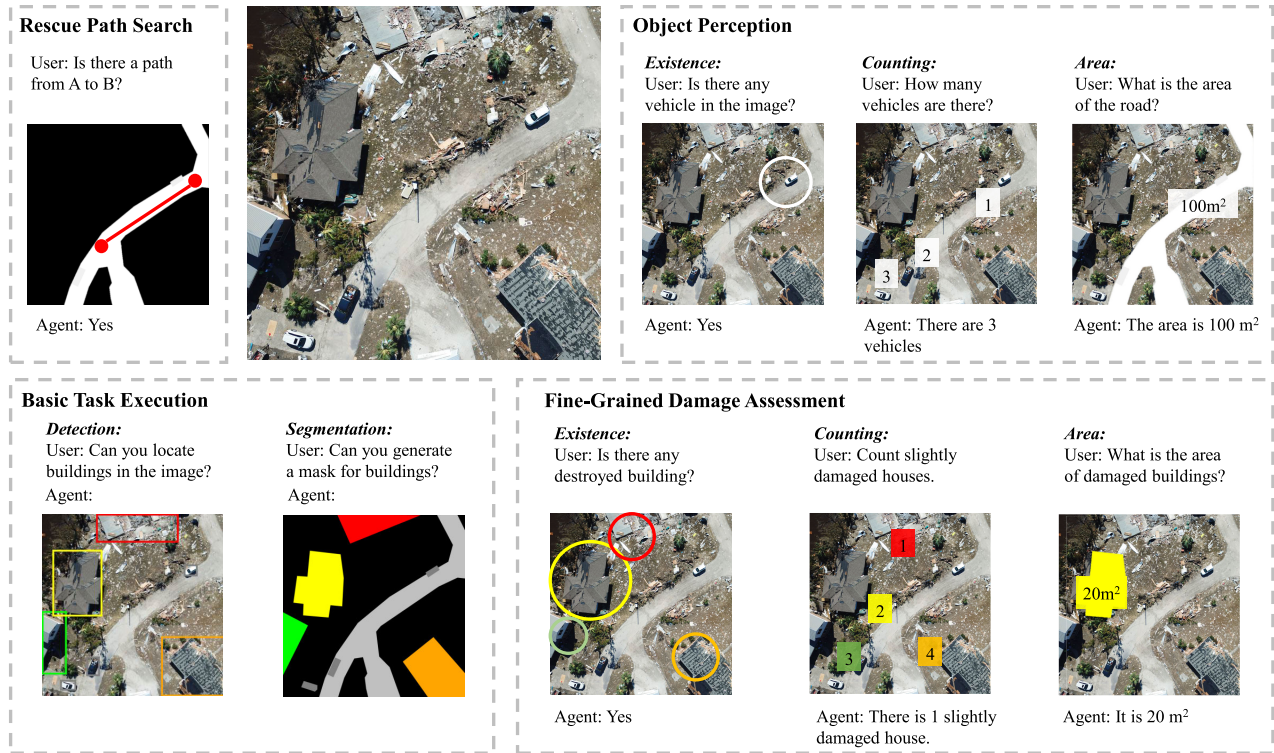


Fig. 5. Demonstration of different types of annotated requests in RescueADI.

Fine-grained damage assessment involves similar tasks but focuses on damage-specific details. It includes determining the existence of damage, counting the instances of damage, and calculating the area affected by the damage. This detailed analysis helps in understanding the severity and extent of the disaster's impact. Lastly, rescue path search is crucial in disaster response as it involves discovering a safe and efficient path between given points. This task addresses the challenge of navigating complex and shifting environments, which has not been adequately covered by existing datasets.

To ensure the diversity of the request text and avoid the possibility of models overfitting to a specific input pattern, requests are created in three steps. Firstly, for each image, we generate requests of different types with a fixed template, which we refer to as the “seed request.” After that, we employ the power of pretrained LLMs to expand the diversity of seed requests. For each request, an LLM is prompted to generate and randomly sample three different forms of the request while keeping the meaning of the request unchanged. As an LLM does not always produce accurate answers, we manually screen the dataset to filter out requests that have been rewritten in wrong ways. During the manual screening process, we carefully review each generated request to ensure that the rephrasing remains faithful to the original intent.

### C. Annotations

The unique structure of the ADI task necessitates a comprehensive annotation approach. Each request is annotated as a tuple of three elements ( $P, R, A$ ), representing planning, perception, and recognition. The planning annotation is detailed

through a list of sub-tasks required to fulfill the request. The perception annotation includes a semantic segmentation map and bounding boxes for objects in the scene. The recognition annotation provides a textual response to the input request. To construct the dataset with these annotations, we employed a combination of manual annotation, automated processes, and existing datasets, ensuring that the resulting annotations are of high quality.

The planning annotation is assigned to requests based on their types during the request generation stage. Although the text of each request is altered during the rephrasing process, the core intention remains consistent with its seed request. Consequently, the required sub-tasks are uniform across requests of the same type, and we identify these sub-tasks to assign the planning annotation accurately.

The perception annotation process includes both segmentation and object detection annotations. The RescueNet dataset provides segmentation annotations for ten categories, including four distinct levels of building damage, meeting the requirements for segmentation tasks. However, the object detection annotation is not provided by the original dataset. To address this, we initially identify the outer bounding box for each connected component in the segmentation map of foreground objects to generate preliminary object detection annotations. These annotations are then visualized, and any inaccuracies are manually adjusted to produce accurate object detection labels for each input image. This procedure yielded 14 483 high-quality annotations of foreground objects in disaster scenes.

For the recognition annotation, as manually annotating textual answers for each image and request is expensive,

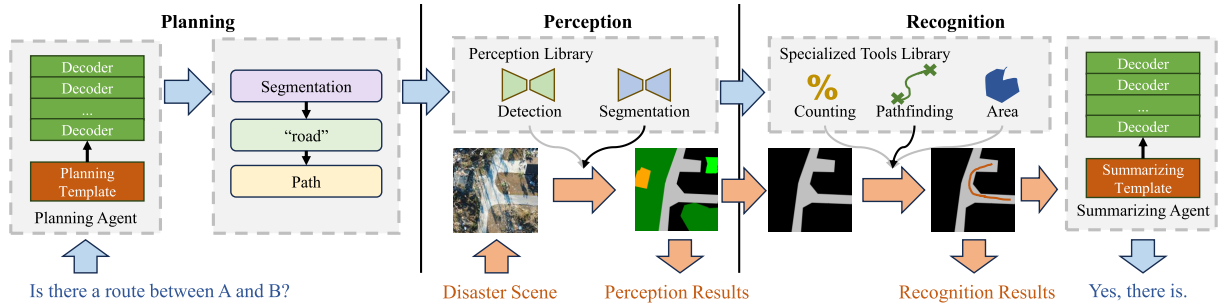


Fig. 6. Overall architecture of our proposed agent-based ADI. The inference process can be divided into planning, perception, and recognition. The stacked decoder represents the LLM constructed with transformer decoders.

we developed a process to synthesize the needed annotations from existing labels with a semi-automatic pipeline. Different methods are employed for the automated part of each request type, as detailed below.

- 1) *Object Presence/Damage Level Presence*: An object presence request asks to identify if there are any objects of a given class in the scene. Therefore, we utilize the segmentation label to produce the ground truth of object presence requests. The answer is “yes” if the segmentation map of a given class is not empty, and “no” otherwise.
- 2) *Object Counting/Damage Level Counting*: A counting request requires an agent to give the exact number of objects of a given class in the scene. We use the object detection label to produce the recognition ground truth of counting requests.
- 3) *Area Calculation*: In the area calculation task, the agent is required to estimate the area of a specified class present in the scene. To achieve this, we use the segmentation label in combination with the ground sample distance (GSD) of the original image. This allows us to calculate the actual area corresponding to the mask of the specified class.
- 4) *Rescue Path Search*: In the rescue path search request, the agent needs to find if an unblocked route exists between given points. The start point and the destination point are manually annotated. To produce the label for this task, we utilize the segmentation label for clear roads and perform the A\* path-finding algorithm to determine if the destination is reachable.

The annotation process for the RescueADI dataset involves a careful combination of manual and automated methods to ensure high-quality and comprehensive annotations. This multistep approach addresses the unique requirements of planning, perception, and recognition annotations, thereby supporting the diverse needs of the ADI task.

## V. AGENT-BASED DISASTER INTERPRETATION

As illustrated in Fig. 6, we introduce a novel framework for agent-based ADI. This approach covers planning, perception, and recognition functionalities through interactions with a set of specialized tools. The agent consists of three parts: the planner module, the perception module, and the recognition module. They are responsible for predicting the best

plan to accomplish the request, providing perception results that supports the plan, and generating responses based on perception results, respectively. Different from existing end-to-end VQA models, the autonomous agent paradigm employs separate neural networks for each part of the agent, which are not connected by gradient flows during training. This design brings more efficient utilization of pretrained large models and offers greater transparency in the decision-making process. Moreover, the use of specialized tools enhances the ability to solve numerical tasks, which are hard for traditional end-to-end models.

### A. Planning Module

To instruct the LLM to output a valid plan that can be executed automatically, we construct the input prompt with the following parts.

- 1) *Task Definition*: The task definition asks the planning agent to create an action plan based on user requests.
- 2) *Format Constraints*: The format constraints demonstrate the required JSON format for the LLM so that the output of the LLM can be parsed automatically. The expected format is an array of JSON objects, where each object represents a single action. An action includes the ID of the tool to use the inputs and outputs of the tool.
- 3) *Tool Descriptions*: The tool descriptions include a list of available tools. For each tool, a brief description is provided and the numbers and types of the inputs and outputs of the tool are given.
- 4) *Input Descriptions*: The descriptions of the input image, including its identifier and resolution.
- 5) *User Request*: The request is concatenated at the end of the prompt.

The planning agent uses an LLM to process the described prompt as input and produces a formatted JSON in the form of a list of actions to execute. The LLM also assigns a unique identifier to each input and output, which is used as a key for storing intermediate results in a dictionary for later reference. As shown in Fig. 7, when dealing with a path-finding problem, the LLM schedules a path-finding tool after performing segmentation. The output mask of the segmentation step is assigned the same identifier as the input mask of the path-finding step, indicating that the path-finding tool takes the output of the segmentation model as its input. The dictionary of intermediate results thus acts as both a data repository and



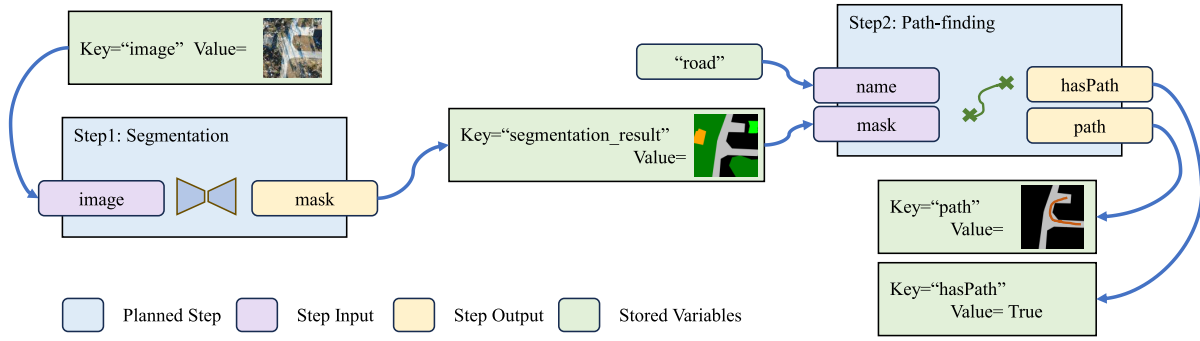


Fig. 7. Specific example of the interaction between the perception and recognition tools when processing a path-finding request. The intermediate results are stored in a dictionary structure to model dependencies between tools.

a reference map, enabling the agent to reuse results and avoid redundant computations.

In practice, the LLM does not always generate a response in the correct format. We design a rejection sampling-based approach to handle this issue. The LLM is prompted to put the desired JSON at the end of its response. A reverse search algorithm is then used to find the valid JSON structure. The search range starts from the last character in the response string. If the substring at the end of the response is not a valid JSON, the beginning of the searching range moves backward until it reaches the start of the response. If no valid JSON is found after the search, we generate the response again with a larger temperature to get a different result until a valid JSON is found.

Once a valid action plan in JSON format is generated, we check the validity of each action in the plan. An action is considered invalid if the tool ID does not exist or the input resource ID is not available. We find that invalid actions can be classified into two categories: typographical error and hallucination. A typographical error happens when the planner agent tries to use an existing tool but generates the wrong tool ID by mistake. Hallucination, on the other hand, is when the LLM invents a tool that does not exist. We identify the two categories by computing the edit distance between the LLM-generated identifier and each existing identifier. If the closest edit distance is less than 8, the identifier is corrected to the closest match. Actions that do not satisfy this criterion are classified as hallucinations and are therefore discarded.

### B. Perception Module

The perception module provides tools for visual perceptions, bridging the gap between LLM and the image modality in disaster scenes. The module provides two vision actions to extract information from the input image. The detection tool takes an image as input and produces a structured representation of detected objects in JSON format. The segmentation tool takes an image as input and outputs masks of existing categories in the scene.

The object detection and semantic segmentation tasks are well-studied tasks in computer vision. Since the agent-based method has a modular structure, we can utilize existing state-of-the-art models for these tools. To implement the object

detection tool, we employ the model structure of GroundingDINO [56] and train the model on the RescueADI dataset to obtain a standard object detection model that detects interested fine-grained categories in disaster scenes. Subsequently, we convert the detection output into a structured JSON to connect it to the agent framework. Each detected object is represented by an item in an array. An item consists of attributes describing the type of the object and the bounding box coordinates of the object.

To implement the semantic segmentation tool, we utilize the PSPNet and train it on the RescueADI dataset. To adapt the model into the agent framework, we train the model on RescueADI and convert the output into a dictionary of binary masks, where the key represents the category and the value represents the mask of the corresponding category.

### C. Recognition Module

The recognition module provides specialized tools to perform recognition of the perception result and utilize an LLM-based agent to perform analysis and produce the final result.

When it comes to numerical tasks, LLMs are prone to mistakes due to the nature of token-based number prediction. To address this issue, we introduce the counting tool, which provides accurate counting results based on object detection outputs. This tool accepts the detection result and a target object type as inputs and returns the exact number of objects of that type.

For dense prediction tasks such as segmentation, it is hard to directly make use of the segmentation mask with a language model. Therefore, we provide the agent with the segmentation area tool. This tool computes the total area covered by a given category from the segmentation result. It takes the segmentation map and the target category name as inputs, outputting the total area of the specified category.

To solve the path-finding problem, we integrate the A-star algorithm into the agent framework as a callable tool. The tool accepts two points and a binary mask as input and finds out if there is a path between the two points. By calling the path-finding tool, the agent can get accurate results to determine if the rescuers can reach a specific destination.

After recognizing the results of the perception, a summarizing agent integrates the output of these tools into its final

answer. The prompt for the summarizing agent contains the following parts.

- 1) *Task Definition*: Requests the agent to provide a final answer based on the user's request.
- 2) *Action History*: Details the sequence of actions performed and their outcomes.
- 3) *User Request*: The original request made by the user.

Through the coordination of these modules, the agent is able to leverage specialized tools to generate accurate, transparent, and reliable outputs for disaster response tasks.

## VI. EXPERIMENTS

In this section, we conduct experiments on our RescueADI dataset using the proposed agent-based method to validate the effectiveness of our task. We also experiment with existing methods to showcase the unique capability of the novel task form.

### A. Evaluation Metrics

The performance of methods on ADI is evaluated from three aspects: planning, perception, and recognition. We evaluate the planning result with VR, recall, and precision. The planning agent may fail to produce a valid plan at all. Therefore, we define the VR as the number of valid plans divided by the total number of requests in the validation set as follows:

$$VR = \frac{|\text{valid}|}{N} \quad (1)$$

where  $|\text{valid}|$  is the number of valid plans and  $N$  is the number of requests. For valid plans, we define precision and recall by referring to the plan as a binary classification problem.

$$P = \frac{CA}{CA + UA} \quad (2)$$

$$R = \frac{CA}{CA + MA} \quad (3)$$

where CA represents the number of correct actions predicted, UA represents the number of unnecessary actions predicted, and MA represents the number of missing actions that are necessary but not predicted.

To evaluate the perception and recognition performance, we divide the requests into two groups according to their type. For requests that involve visual perception tasks, the agent outputs object detection or segmentation results, which are evaluated with mean IoU (mIoU) and mAP.

For question-answering results, we adopt two different metrics to obtain a more comprehensive evaluation. The matching rate is calculated by strictly matching the ground truth with the predicted result. For area calculation requests, which require a numerical answer, we match the generated number and the ground truth. An answer is considered correct if the difference is less than one square meter or the relative difference is less than two percent. For other types of requests, the answer must match the exact ground truth to be considered correct. However, in the context of natural language, this strict matching rule may eliminate answers that are semantically correct but in a different form. Therefore, we adopt GPTScore following

HuggingGPT [58] to provide a more flexible judgment to the answers. GPTScore asks an LLM to judge the correctness of an answer given the ground truth. This metric takes into consideration the semantics of the answer and produces results closer to human interpretation. Specifically in ADI tasks, there could be multiple answer variants that are correct. For example, if the agent responds with “*There are no houses in the image*” while the ground truth is simply “0,” the matching rule may fail to recognize the two answers that are of the same meaning, while GPTScore can. Therefore, both metrics are presented to ensure a comprehensive evaluation of the inference results.

### B. Implementation Details

1) *Large Language Model*: The modular framework of our method enables us to experiment with different LLMs without the need to retrain other parts of the agent. In our experiments, we use the close-sourced GPT-4o-mini as the backend of the planning and summarizing agent. In the ablation studies, we also experiment with state-of-the-art open-source LLMs. When generating the output with LLMs, we adopt the greedy decoding strategy with a temperature of 0.7.

2) *Transformer Decoder*: The decoder structure illustrated in Fig. 6 refers to the decoder-only Generative Pretrained Transformers structure commonly adopted by a wide range of state-of-the-art LLMs [14], [17], [18], [19]. A decoder consists of positional embedding, multihead attention, and feed-forward network (FFN) layers. The positional embeddings are designed to inject sequence order information into input sequences. The multihead self-attention layer allows the information to be exchanged between different positions in the sequence. The FFN applies position-wise transformations to each token's representation.

3) *Perception Models*: The perception models are trained on four RTX3090 graphic cards. The PSPNet and FasterRCNN are initialized with pretrained ResNet-50 [63] backbones and trained with the SGD optimizer. The SAN and GroundingDINO are initialized with pretrained ViT-B [64] backbones and trained with the AdamW optimizer.

4) *Specialized Tools*: The specialized tools in our agent-based disaster interpretation method are implemented in the form of Python functions with multiple inputs and outputs. Specifically, the tools come with additional annotations describing the inputs and outputs of each tool, along with their types, to seamlessly integrate them into the autonomous agent's workflow. During inference, intermediate results generated by these tools are stored in a dictionary data structure, where each result is associated with a unique textual identifier serving as its key. This design enables the LLM to manipulate various formats, such as texts, images, or masks. An automated script verifies the outputs of the planning agent against tool definitions. If the plan is valid, the corresponding Python functions are executed with the provided inputs, and their outputs are converted into strings. These stringified outputs are then passed to the summarizing agent to produce the final response.

In our experimental setup, we included tools such as the area calculation tool, the counting tool, and the path-finding

TABLE II

QUALITATIVE COMPARISON BETWEEN ADI AND EXISTING TASKS ON RESCUEADI. THE OUTPUT FORMS THAT EXISTING TASK FORMS CANNOT PRODUCE ARE MARKED AS EMPTY CELLS. THE RECOGNITION ACCURACY HIGHLIGHTS THE EFFECTIVENESS OF THE PROPOSED METHOD COMPARED TO EXISTING LARGE VISION-LANGUAGE MODELS

| Task         | Model                         | Planning |        |        | Perception |                   | Recognition   |               |
|--------------|-------------------------------|----------|--------|--------|------------|-------------------|---------------|---------------|
|              |                               | VR       | P      | R      | mIoU       | mAP <sub>50</sub> | Exact         | GPTScore      |
| Segmentation | SAN [60]                      | -        | -      | -      | 0.6802     | -                 | -             | -             |
|              | PSPNet [61]                   | -        | -      | -      | 0.6828     | -                 | -             | -             |
| Detection    | FasterRCNN [62]               | -        | -      | -      | -          | 0.7640            | -             | -             |
|              | GroundingDINO [56]            | -        | -      | -      | -          | 0.8010            | -             | -             |
| VQA          | GeoChat [45]                  | -        | -      | -      | -          | -                 | 0.2263        | 0.2975        |
|              | VisualGLM [42]                | -        | -      | -      | -          | -                 | 0.6535        | 0.6915        |
| ADI          | Ours w/ PSPNet, GroundingDINO | 0.9960   | 0.9105 | 0.9728 | 0.6828     | 0.8010            | <b>0.7563</b> | <b>0.7896</b> |

tool. These were chosen because they address core tasks that are broadly applicable across diverse disaster scene interpretation problems. In addition, the agent's design allows further extensibility. New tools can be added to the system to address emerging requirements by defining new functions with appropriate annotations, thereby enhancing its generalizability in evolving disaster scenarios.

### C. Quantitative Analysis

We evaluate the proposed method on the RescueADI dataset. As summarized in Table II, we compare the capability of our agent-based method with existing models across three critical aspects: planning, perception, and recognition. Notably, traditional object detection and semantic segmentation methods only produce perception results while the VQA methods focus solely on producing recognition results. In contrast, our method addresses all three dimensions, demonstrating its comprehensive capabilities.

1) *Planning Performance*: Planning is the most important stage in ADI as the proper usage of tools is crucial to the interpretation results. Among all related tasks, our framework is the only one that produces explicit planning outputs. The quality of the planning stage is evaluated with VR, precision, and recall. A high VR indicates that the proposed agent produces a reliable planning format instead of outputting corrupted text.

With GPT-4o-mini as the LLM backend, our agent-based approach consistently produces valid plans in over 99% of cases. The planning achieves a precision of 90.51% and a recall of 97.24%, showing the robustness and effectiveness of the agent-based framework.

2) *Perception Performance*: As our approach utilizes state-of-the-art perception models as a part of its tool library, the accuracy of the final answer is linked to the performance of these models. Therefore, we conduct experiments with different detection and segmentation models on RescueADI dataset and select PSPNet [61] and GroundingDINO [56] as the most accurate and reliable tools.

To evaluate the perception performance, we compute metrics directly on the raw segmentation and detection labels for more comprehensive results. To be specific, the perception scores are calculated in a way that is independent of planning

results. In our proposed framework, the perception models are scheduled by the planning module, and the perception outputs are directly fed to the recognition model. Therefore, our proposed method inherits the accuracy of the segmentation model and the detection model used to build the tool library, achieving a 68.28% mIoU for segmentation and an 80.10% AP for object detection.

3) *Recognition Performance*: The recognition metrics demonstrate the advantage of our agent-based method over end-to-end LLM-based VQA methods. With the utilization of explicit perception models as tools, the agent produces more accurate answers, offering intermediate outputs such as image segmentation and object detection to enhance decision-making processes. We compare the agent-based method with two baseline methods, VisualGLM [42] and GeoChat [45]. Since VisualGLM is primarily developed for natural scenes instead of remote sensing scenarios, we conduct extra tuning with LoRA [65] to align it with the recognition requirements of the RescueADI dataset. Our approach achieves 73.74% exact-match accuracy and 80.90% GPTScore, which is superior compared to both tuned VisualGLM and GeoChat.

To better understand the advantage of the agent-based framework, we inspect the planning and recognition accuracy for each individual request type. As shown in Table III, end-to-end VQA models tend to make mistakes on numerical requests such as counting and area calculations as they do not explicitly produce perception results of the image. Equipped with dedicated perception modules and specialized tools, our agent-based approach exhibits strong accuracy for numerical answers. In our approach, given accurate enough perception results, the specialized tools can perform counting and area calculation without error. The agent automatically determines the perception models to use and then selects specialized tools to perform counting and area calculation tasks accurately. For rescue path-finding requests, the agent also performs better as the task, which is difficult to model by end-to-end neural networks, is broken into simpler perception steps and tool calls by the agent.

### D. Ablation Studies

1) *LLM Backend*: The proposed agent-based framework is designed to be compatible with different LLM backends as



TABLE III

DETAILED COMPARISON OF THE ANSWER ACCURACY BY REQUEST TYPES. IN THE TABLE, *dmg* DENOTES THE ACCURACY OF RECOGNIZING FINE-GRAINED DAMAGE LEVELS WHILE *obj* DENOTES THE ACCURACY OF OTHER OBJECTS

|                | existence     |               | counting      |               | area          |               | path          | all           |
|----------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
|                | obj           | dmg           | obj           | dmg           | obj           | dmg           |               |               |
| GeoChat [45]   | 0.4192        | 0.3265        | 0.0345        | 0.1453        | 0.2222        | 0.3000        | 0.4744        | 0.2975        |
| VisualGLM [42] | 0.6527        | 0.6633        | 0.8103        | 0.7436        | 0.3333        | 0.3500        | 0.8462        | 0.6535        |
| Ours           | <b>0.8024</b> | <b>0.8367</b> | <b>0.8621</b> | <b>0.8462</b> | <b>0.7222</b> | <b>0.4333</b> | <b>0.8846</b> | <b>0.7896</b> |

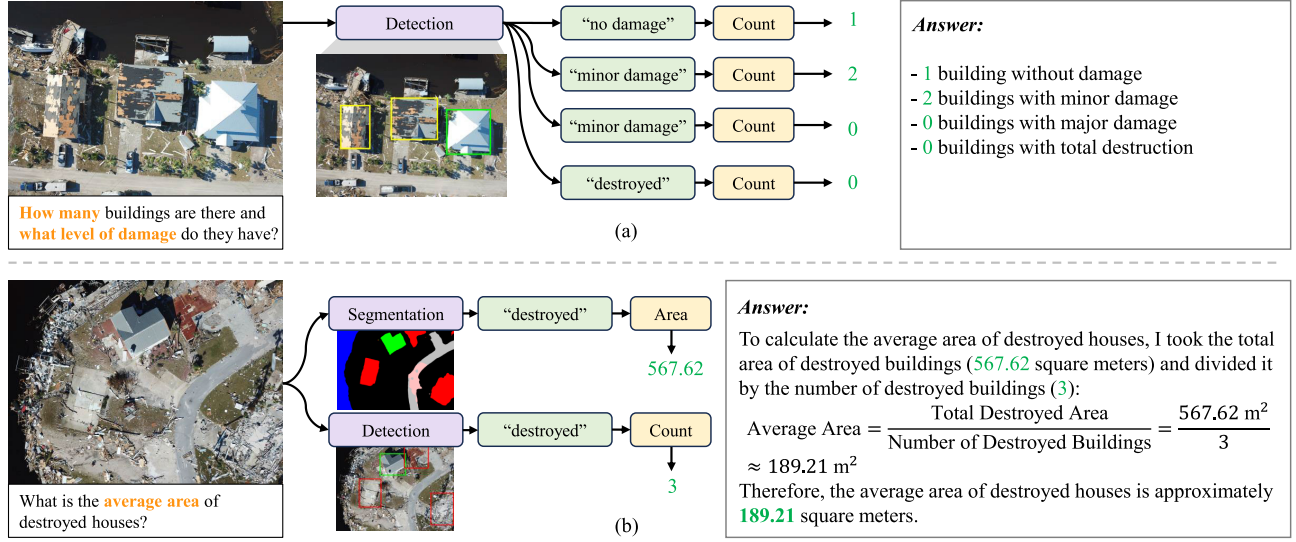


Fig. 8. Visualization of the planning, perception, and recognition results of the proposed method on complex requests. (a) Compound request. (b) Request that requires advanced reasoning.

TABLE IV

ABLATION STUDY ON LLM BACKENDS

| Model               | Planning      |               |               | Recognition   |               |
|---------------------|---------------|---------------|---------------|---------------|---------------|
|                     | VR            | P             | R             | Exact         | GPTScore      |
| Ours w/ GPT 4o mini | <b>0.9960</b> | 0.9105        | <b>0.9728</b> | <b>0.7563</b> | <b>0.7848</b> |
| Ours w/ Deepseek 7b | 0.2615        | 0.5781        | 0.6667        | 0.0728        | 0.1297        |
| Ours w/ Qwen1.5 7b  | 0.6603        | 0.7932        | 0.8558        | 0.3006        | 0.3861        |
| Ours w/ Qwen2.5 7b  | 0.9890        | <b>0.9573</b> | 0.6586        | 0.5411        | 0.6867        |
| Ours w/ Llama3.1 8b | 0.8537        | 0.6629        | 0.9554        | 0.5269        | 0.4937        |

long as the LLM is tuned to follow input instructions. To better understand the effect of different LLM backends, we experiment with several state-of-the-art instruction-following LLMs: GPT-4o-mini, Deepseek 7B, Qwen1.5 7B, Qwen2.5 7B, and Llama 3.1 8B. The LLMs are inserted into the framework and act as the text generator of both the planning agent and the summarizing agent. Results in Table IV show that the capability of the LLM backends has a strong correlation to the planning quality of the agent and, therefore impacts the subsequent perception and recognition. The closed-source LLM, GPT-4o-mini, exhibits the best capability while the best open-source LLM for the task is Qwen2.5 7B. The planning precision of Qwen2.5 7B is slightly higher than that of GPT-4o-mini, but the recall is significantly lower, resulting in reduced overall accuracy. Therefore, we conclude that a strong LLM backend is crucial in the proposed agent-based framework.

TABLE V

ABLATION STUDY ON SPECIALIZED TOOLS

| Model                      | Planning      |               |               | Recognition   |               |
|----------------------------|---------------|---------------|---------------|---------------|---------------|
|                            | VR            | P             | R             | Exact         | GPTScore      |
| Ours w/ specialized tools  | 0.9960        | <b>0.9105</b> | <b>0.9728</b> | <b>0.7563</b> | <b>0.7848</b> |
| Ours w/o specialized tools | <b>0.9990</b> | 0.8556        | 0.9448        | 0.5332        | 0.6092        |

2) *Specialized Tools*: In our proposed method, we employ specialized tools in the recognition module to help analyze the perception result. These tools cover counting, area calculation, and pathfinding. To investigate the contribution of these tools, we perform experiments without specialized tools as shown in Table V. As specialized tools are removed, we observe a significant drop in recognition accuracy. Both exact match score and GPTScore drop by more than 15%, indicating that specialized tools play an important role in ADI.

3) *Performance Boundaries*: Powered by pretrained LLMs, the proposed agent-based method is able to generalize toward more complex requests. As shown in Fig. 8(a), the agent can handle compound questions that require the perception of multiple categories. The request asks to count the number of buildings in different damage levels and the planning agent successfully gives a reasonable plan that uses the counting tool to process each damage level. Fig. 8(b) shows a request that requires advanced reasoning. The agent performs both segmentation and detection to obtain the total area and the

number of buildings that are destroyed and utilizes these numbers to calculate the correct answer.

## VII. CONCLUSION

In this article, we propose a novel ADI task to address the problem of extra human intervention and lack of accuracy in current disaster interpretation pipelines. On top of that, we produce the RescueADI dataset, the first dataset for ADI, that integrates planning, perception, and recognition of disaster scenarios, serving as a robust benchmark. We propose the first autonomous agent-based method with specialized tools to solve the challenging disaster interpretation task. Experiment results prove the effectiveness of the agent-based framework on ADI task, showing an accuracy improvement of more than 9% compared to existing VQA methods. However, experiments have also indicated that the capability of the LLM backend can be crucial to the overall results. Our future work will be focused on improving the robustness of the whole framework and further expanding the variety of the dataset.

## REFERENCES

- [1] D. Brunner, G. Lemoine, and L. Bruzzone, "Earthquake damage assessment of buildings using VHR optical and SAR imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 5, pp. 2403–2420, May 2010.
- [2] N. Kussul, M. Lavreniuk, S. Skakun, and A. Shelestov, "Deep learning classification of land cover and crop types using remote sensing data," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 5, pp. 778–782, May 2017.
- [3] B. Huang, B. Zhao, and Y. Song, "Urban land-use mapping using a deep convolutional neural network with high spatial resolution multispectral remote sensing imagery," *Remote Sens. Environ.*, vol. 214, pp. 73–86, Sep. 2018.
- [4] Y. Shen et al., "BDANet: Multiscale convolutional neural network with cross-directional attention for building damage assessment from satellite images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5402114.
- [5] M. S. Abdalzaher, M. S. Soliman, and S. M. El-Hady, "Seismic intensity estimation for earthquake early warning using optimized machine learning model," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5914211.
- [6] M. Rahmemonfar, T. Chowdhury, and R. Murphy, "RescueNet: A high resolution UAV semantic segmentation dataset for natural disaster damage assessment," *Sci. Data*, vol. 10, no. 1, p. 913, Dec. 2023.
- [7] C. Kyrkou and T. Theodoridis, "EmergencyNet: Efficient aerial image classification for drone-based emergency monitoring using atrous convolutional feature fusion," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 1687–1699, 2020.
- [8] R. Gupta et al., "Creating xBD: A dataset for assessing building damage from satellite imagery," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPR)*, Jun. 2019, pp. 10–17.
- [9] Y. Bai et al., "Pyramid pooling module-based semi-Siamese network: A benchmark model for assessing building damage from xBD satellite imagery datasets," *Remote Sens.*, vol. 12, no. 24, p. 4055, Dec. 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:230526541>
- [10] Y. Zhao, G. Sun, Z. Ling, A. Zhang, and X. Jia, "Point-based weakly supervised deep learning for semantic segmentation of remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5638416.
- [11] W. Zhan et al., "Simultaneous prediction of petrophysical properties and formation layered thickness from acoustic logging data using a modular cascading residual neural network (MCARNN) with physical constraints," *J. Appl. Geophys.*, vol. 224, May 2024, Art. no. 105362.
- [12] S. Antol et al., "VQA: Visual question answering," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 2425–2433.
- [13] A. Sarkar, T. Chowdhury, R. R. Murphy, A. Gangopadhyay, and M. Rahmemonfar, "SAM-VQA: Supervised attention-based visual question answering model for post-disaster damage assessment on remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4702716. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258726537>
- [14] W. Xin Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
- [15] J. Zhang, J. Huang, S. Jin, and S. Lu, "Vision-language models for vision tasks: A survey," 2023, *arXiv:2304.00685*.
- [16] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating LLM hallucination via self reflection," in *Proc. Findings Assoc. Comput. Linguistics, EMNLP*, 2023, pp. 1827–1843.
- [17] T. B. Brown et al., "Language models are few-shot learners," 2020, *arXiv:2005.14165*.
- [18] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [19] J. Bai et al., "Qwen technical report," 2023, *arXiv:2309.16609*.
- [20] P. Maes, "Modeling adaptive autonomous agents," *Artif. Life*, vol. 1, nos. 1–2, pp. 135–162, Oct. 1993.
- [21] L. Wang et al., "A survey on large language model based autonomous agents," *Frontiers Comput. Sci.*, vol. 18, no. 6, Dec. 2024, Art. no. 186345.
- [22] N. M. Saadi, M. A. Zaher, F. El-Baz, and K. Watanabe, "Integrated remote sensing data utilization for investigating structural and tectonic history of the Ghadames basin, Libya," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 13, no. 5, pp. 778–791, Oct. 2011.
- [23] Y. Feng, R. I. Negrón-Juárez, and J. Q. Chambers, "Remote sensing and statistical analysis of the effects of hurricane María on the forests of Puerto rico," *Remote Sens. Environ.*, vol. 247, Sep. 2020, Art. no. 111940. [Online]. Available: <https://www.science-direct.com/science/article/pii/S0034425720303102>
- [24] M. Rahmemonfar, T. Chowdhury, A. Sarkar, D. Varshney, M. Yari, and R. R. Murphy, "FloodNet: A high resolution aerial imagery dataset for post flood scene understanding," *IEEE Access*, vol. 9, pp. 89644–89654, 2021.
- [25] X. Zhu, J. Liang, and A. Hauptmann, "MSNet: A multilevel instance segmentation network for natural disaster damage assessment in aerial videos," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, Jan. 2021, pp. 2023–2032.
- [26] S. Lobry, D. Marcos, J. Murray, and D. Tuia, "RSVQA: Visual question answering for remote sensing data," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 12, pp. 8555–8566, Dec. 2020, doi: [10.1109/TGRS.2020.2988782](https://doi.org/10.1109/TGRS.2020.2988782).
- [27] X. Yuan, J. Shi, and L. Gu, "A review of deep learning methods for semantic segmentation of remote sensing imagery," *Expert Syst. Appl.*, vol. 169, May 2021, Art. no. 114417.
- [28] J. Wang, Z. Zheng, A. Ma, X. Lu, and Y. Zhong, "LoveDA: A remote sensing land-cover dataset for domain adaptive semantic segmentation," 2021, *arXiv:2110.08733*.
- [29] D. Zhao et al., "Panoptic perception: A novel task and fine-grained dataset for universal remote sensing image interpretation," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5620714.
- [30] H. Chen and Z. Shi, "A spatial-temporal attention-based method and a new dataset for remote sensing image change detection," *Remote Sens.*, vol. 12, no. 10, p. 1662, May 2020. [Online]. Available: <https://www.mdpi.com/2072-4292/12/10/1662>
- [31] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, Mar. 2023.
- [32] T. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, Zurich, Switzerland. Cham, Switzerland: Springer, Sep. 2014, pp. 740–755.
- [33] S. Shao et al., "Objects365: A large-scale, high-quality dataset for object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 8430–8439.
- [34] A. Kuznetsova et al., "The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale," *Int. J. Comput. Vis.*, vol. 128, no. 7, pp. 1956–1981, Jul. 2020.
- [35] W. Zhang, P. Tang, and L. Zhao, "Remote sensing image scene classification using CNN-CapsNet," *Remote Sens.*, vol. 11, no. 5, p. 494, 2019.
- [36] K. Nogueira, O. A. B. Penatti, and J. A. Dos Santos, "Towards better exploiting convolutional neural networks for remote sensing scene classification," *Pattern Recognit.*, vol. 61, pp. 539–556, Jan. 2017.
- [37] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.
- [38] L. Zhou, H. Palangi, L. Zhang, H. Hu, J. Corso, and J. Gao, "Unified vision-language pre-training for image captioning and VQA," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 7, pp. 13041–13049.

- [39] K. Marino, M. Rastegari, A. Farhadi, and R. Mottaghi, "OK-VQA: A visual question answering benchmark requiring external knowledge," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3195–3204.
- [40] S. Lu, Y. Ding, M. Liu, Z. Yin, L. Yin, and W. Zheng, "Multiscale feature extraction and fusion of image and text in VQA," *Int. J. Comput. Intell. Syst.*, vol. 16, no. 1, p. 54, Apr. 2023.
- [41] C. Feng, D. Danier, F. Zhang, and D. Bull, "RankDVQA: Deep VQA based on ranking-inspired hybrid training," in *Proc. IEEE Winter Conf. Appl. Comput. Vis.*, Jun. 2024, pp. 1648–1658.
- [42] M. Ding et al., "Cogview: Mastering text-to-image generation via transformers," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 34, 2021, pp. 19822–19835.
- [43] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," 2023, *arXiv:2304.08485*.
- [44] C. Chappuis, V. Zermatten, S. Lobry, B. Le Saux, and D. Tuia, "Prompt-RSVQA: Prompting visual context to a language model for remote sensing visual question answering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2022, pp. 1371–1380.
- [45] K. Kuckreja, M. S. Danish, M. Naseer, A. Das, S. Khan, and F. S. Khan, "GeoChat: Grounded large vision-language model for remote sensing," 2023, *arXiv:2311.15826*.
- [46] D. Zhao, J. Lu, and B. Yuan, "See, perceive, and answer: A unified benchmark for high-resolution postdisaster evaluation in remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5618914.
- [47] T. Guo et al., "Large language model based multi-agents: A survey of progress and challenges," 2024, *arXiv:2402.01680*.
- [48] S. Qiao et al., "AutoAct: Automatic agent learning from scratch for QA via self-planning," 2024, *arXiv:2401.05268*.
- [49] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for 'mind' exploration of large language model society," 2023, *arXiv:2303.17760*.
- [50] C. Qian et al., "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 15174–15186.
- [51] Z. Wang, S. Cai, G. Chen, A. Liu, X. S. Ma, and Y. Liang, "Describe, explain, plan and select: Interactive planning with LLMs enables open-world multi-task agents," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–11.
- [52] X. Zhu et al., "Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory," 2023, *arXiv:2305.17144*.
- [53] R. Nakano et al., "WebGPT: Browser-assisted question-answering with human feedback," 2021, *arXiv:2112.09332*.
- [54] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, Jan. 2023, pp. 1–16.
- [55] C. Wu, S. Yin, W. Qi, X. Wang, Z. Tang, and N. Duan, "Visual ChatGPT: Talking, drawing and editing with visual foundation models," 2023, *arXiv:2303.04671*.
- [56] S. Liu et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," 2023, *arXiv:2303.05499*.
- [57] A. Kirillov et al., "Segment anything," 2023, *arXiv:2304.02643*.
- [58] Y. Shen, K. Song, X. Tan, D. Li, W. Lü, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in hugging face," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, Jan. 2023, pp. 1–20.
- [59] W. Xu, Z. Yu, Y. Wang, J. Wang, and M. Peng, "RS-agent: Automating remote sensing tasks through intelligent agents," 2024, *arXiv:2406.07089*.
- [60] M. Xu, Z. Zhang, F. Wei, H. Hu, and X. Bai, "SAN: Side adapter network for open-vocabulary semantic segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 15546–15561, Dec. 2023.
- [61] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2881–2890.
- [62] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., 2015, pp. 1–7. [Online]. Available: [https://proceedings.neurips.cc/paper\\_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf)
- [63] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [64] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [65] J. E. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2021, pp. 1–11. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>